សកលវិទ្យាល័យភូមិន្ទភ្នំពេញ

មហាវិទ្យាល័យវិទ្យាសាស្ត្រ

ដេប៉ាតឺម៉ង់ព័ត៌មានវិទ្យា

សមាជិកក្រុម

១. ជា គីមឡេង

២. ហេង ឡាយ

៣. សំអាង ដារ៉ា

៤. ផាន ដាវីត

៥. ចន វឌ្ឍនា

៦. សៀង ម៉ាលីន

៧. មឿន មករា

ថ្នាក់ A2

មុខវិជ្ជា៖ OOAD and Programming

បង្រៀនដោយលោកគ្រូ៖ Svoeuy Bunna

២០២៥-២០២៦

Contents

Chepter:1 (Composite Pattern)

1. និយមន័យ (Definition)
2. រចនាសម្ព័ន្ធរួម (UML Class Diagram)
3. ឧទាហរណ៍ Composite Pattern ក្នុងជីវិតពិត
4. Example Code (Concept)
5. អត្ថប្រយោជន៍ (Advantages)
6. គុណវិបត្តិ (Disadvantages)
7. ពេលណាគួរប្រើ Composite Pattern
8. ពេលណាមិនគួរប្រើ Composite Pattern

Chepter:2 (Composite Pattern)

1. និយមន័យ (Definition)
2. រចនាសម្ព័ន្ធរួម (UML Class Diagram)
3. គោលបំណង (Purpose)
4. រចនាសម្ព័ន្ធ (Structure)
5. UML Class Diagram របស់ Singleton
6. ដំណើរការរបស់ Singleton Pattern
7. ឧទាហរណ៍ជាក់ស្តែង (Real-world Examples)
8. Example Code (C++)
9. អត្ថប្រយោជន៍ (Advantages)

10. គុណវិបត្តិ (Disadvantages)
11. ពេលណាគួរប្រើ Singleton Pattern
12. ពេលណាមិនគួរប្រើ Singleton Pattern

Chepter:3 (Composite Pattern)

1. និយមន័យ (Definition)
2. រចនាសម្ព័ន្ធរួម (UML Class Diagram)
3. បរចនាសម្ព័ន្ធ (Structure)
4. UML Class Diagram របស់ Observer Pattern
5. ដំណើរការរបស់ Observer Pattern
6. ឧទាហរណ៍ជាក់ស្តែង (Real-world Examples)
7. Example Code (C++)
8. អត្ថប្រយោជន៍ (Advantages)
9. គុណវិបត្តិ (Disadvantages)
10. ពេលណាគួរប្រើ Observer Pattern
11. ពេលណាមិនគួរប្រើ Observer Pattern

Chepter1: Composite Pattern

1. និយមន័យ (Definition)

ក្នុង Object-Oriented Analysis and Design (OOAD) Composite Pattern គឺជាប្រភេទ Structural Design Pattern មួយ ដែលអនុញ្ញាតឲ្យយើងរៀបចំ objects ជាទម្រង់ tree structure (រចនាសម្ព័ន្ធដើមឈើ) ដើម្បីបង្ហាញពីទំនាក់ទំនងផ្នែក និងទាំងមូល (Part–Whole Hierarchy)។

Composite Pattern អនុញ្ញាតឲ្យអ្នកប្រើប្រាស់ (Client) អាចប្រើប្រាស់ Object តែមួយ (Leaf) និង Object ជាក្រុម (Composite) ដោយប្រើ interface ដូចគ្នា។

និយាយឲ្យសាមញ្ញ៖

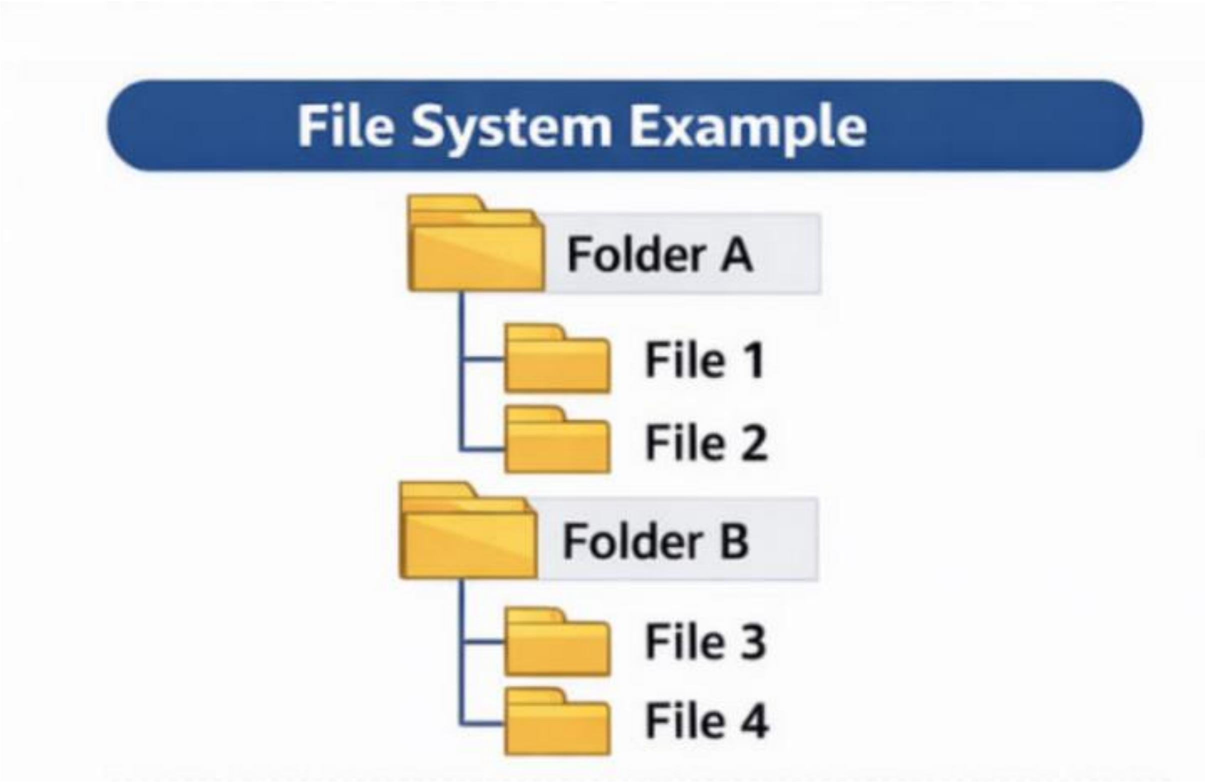
Composite Pattern ធ្វើឲ្យកម្មវិធីអាចដំណើរការលើ object មួយឯកត្តា និង object ជាក្រុម ដូចគ្នា។

+ ឧទាហរណ៍ជាក់ស្តែង ៖

File System ក្នុង Computer

File → Object មួយ (Leaf)

Folder → Object ជាក្រុម (Composite)



2. រចនាសម្ព័ន្ធរួម (UML Class Diagram)

UML Class Diagram មាន 3 ផ្នែកសំខាន់ៗ៖

- Component
- Leaf
- Composite

1. Component

Component គឺជា interface ឬ abstract class ដែលកំណត់ method សម្រាប់ Leaf និង Composite ប្រើដូចគ្នា។

វាជា base class ដែល object ផ្សេងៗត្រូវអនុវត្តតាម។

ឧទាហរណ៍ Method៖

+ add( )

+ remove( )

+ display( )

Component អនុញ្ញាតឲ្យ Client ប្រើ Leaf និង Composite ដូចគ្នា។

2. Leaf

Leaf គឺជា object ដែលតំណាងឲ្យ ជាតុឯកត្តា (Single Object)។

លក្ខណៈសម្បត្តិ៖

មិនអាចមាន object ផ្សេងនៅខាងក្នុងបាន

អនុវត្ត method ពី Component

ជាជាតុចុងក្រោយក្នុង tree structure

ឧទាហរណ៍៖

+ File ក្នុង Folder

+ Employee ម្នាក់ក្នុង Department

+ Button មួយក្នុង GUI

3. Composite

Composite គឺជា object ដែលអាចផ្ទុក Leaf ឬ Composite ផ្សេងទៀត។វាតំណាងឲ្យ Group of Objects។

លក្ខណៈសម្បត្តិ៖

អាចមាន child objects

អនុវត្ត method ពី Component

អាចបន្ថែម ឬដក objects

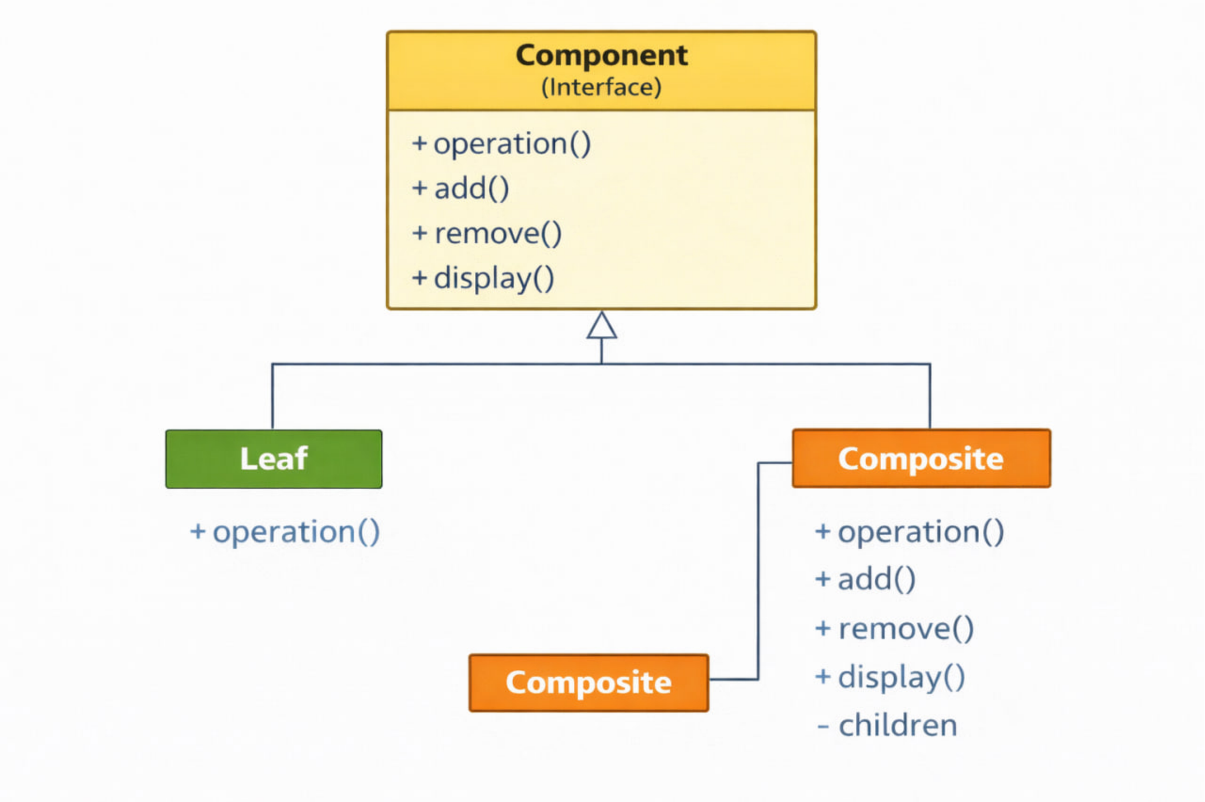
ឧទាហរណ៍៖

+ Folder ដែលមាន Files

+ Department ដែលមាន Employees

+ Panel ដែលមាន Buttons

UML Class Diagram របស់ Composite Pattern



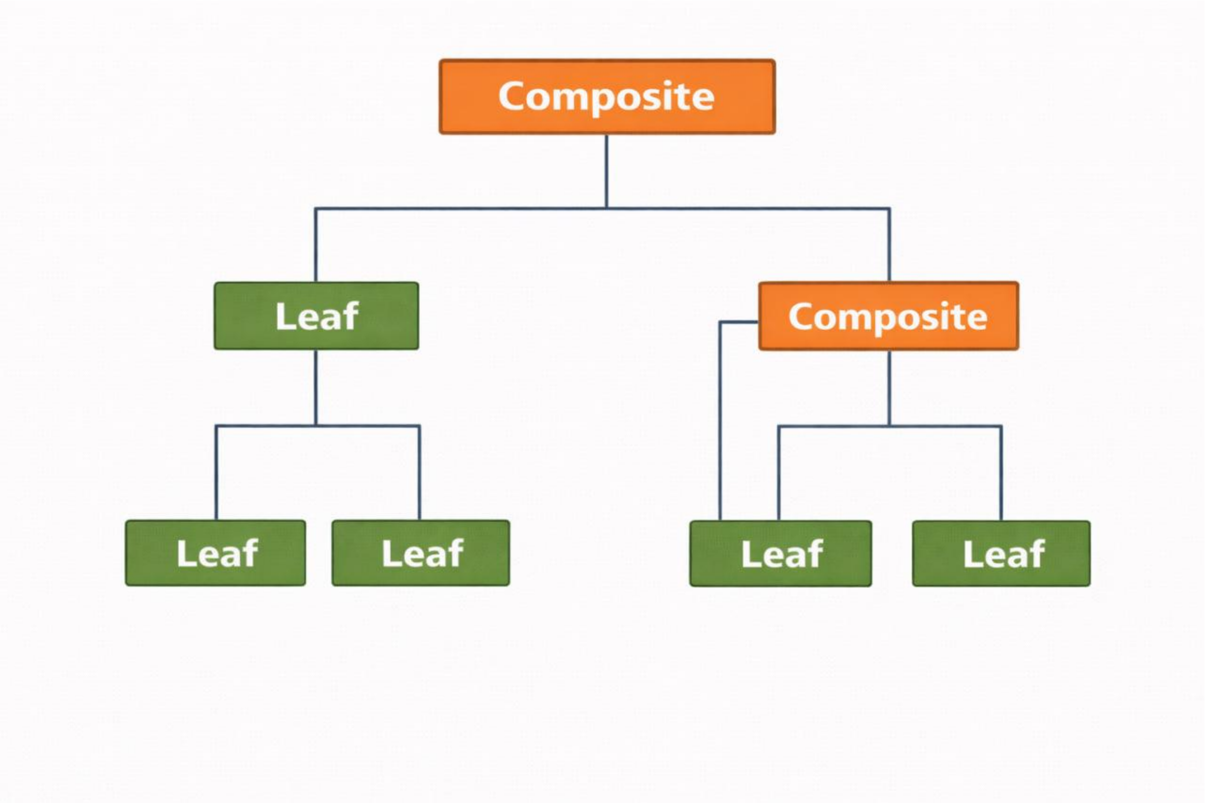
ការពន្យល់ Diagram៖

+ Component ជា base class ដែលកំណត់ operation

+ Leaf គឺ object តែមួយ ដែលមិនមាន children

+ Composite មាន children ដែលអាចជា Leaf ឬ Composite ផ្សេងទៀត។

+ Composite បង្កើត Tree Structure ដូចជា៖



3. ឧទាហរណ៍ Composite Pattern ក្នុងជីវិតពិត

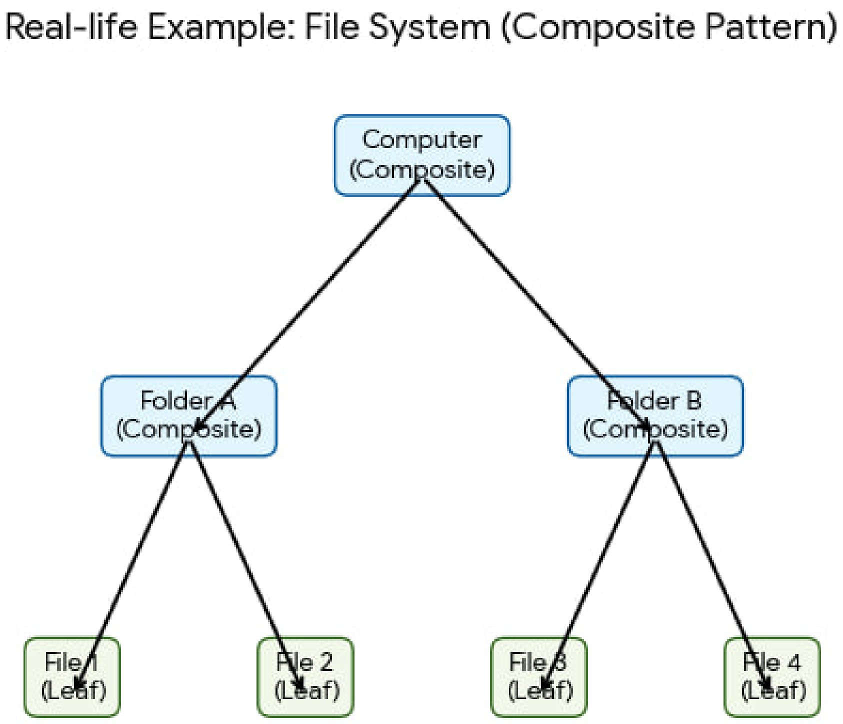
1. File System (ប្រព័ន្ធឯកសារ)

នេះគឺជាឧទាហរណ៍ដែលបង្ហាញពីការរៀបចំវត្ថុជាមែកធាង (Tree Structure)។ ក្នុងកុំព្យូទ័រ អ្នកអាចមានឯកសារទោល (Files) និងថតឯកសារ (Folders)។ ថតឯកសារអាចមានទាំងឯកសារ និងថតឯកសារផ្សេងទៀតនៅខាងក្នុងវា។

Component: FileSystemItem (Interface រួម)

Leaf: File (វត្ថុទោល មិនមានកូន)

Composite: Folder (វត្ថុបណ្តុំ អាចផ្ទុកកូនបាន)



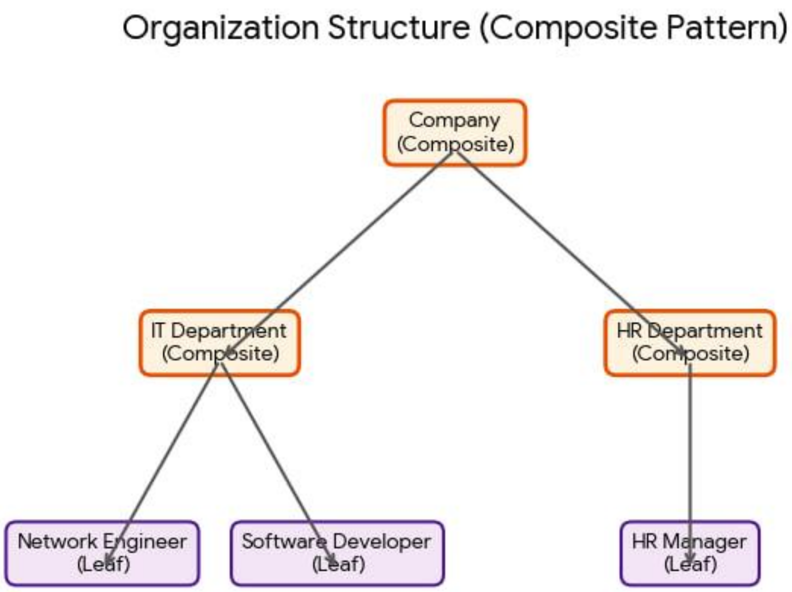
2. Organization Structure (រចនាសម្ព័ន្ធក្រុមហ៊ុន)

ក្នុងក្រុមហ៊ុនមួយ បុគ្គលិកអាចជាអ្នកធ្វើការធម្មតា (Leaf) ឬជាអ្នកគ្រប់គ្រង (Composite) ដែលដឹកនាំក្រុម ឬនាយកដ្ឋាន។ Pattern នេះអនុញ្ញាតឱ្យក្រុមហ៊ុនគណនាទិន្នន័យ (ដូចជា ប្រាក់ខែសរុប) បានយ៉ាងងាយស្រួលទោះជាលើបុគ្គលិកម្នាក់ ឬនាយកដ្ឋានទាំងមូល។

Component: Employee (Interface)

Leaf: Developer, Engineer (បុគ្គលិកគ្មានកូនចៅ)

Composite: Manager, Director (អ្នកដែលមានកូនចៅក្រោមបង្គាប់)



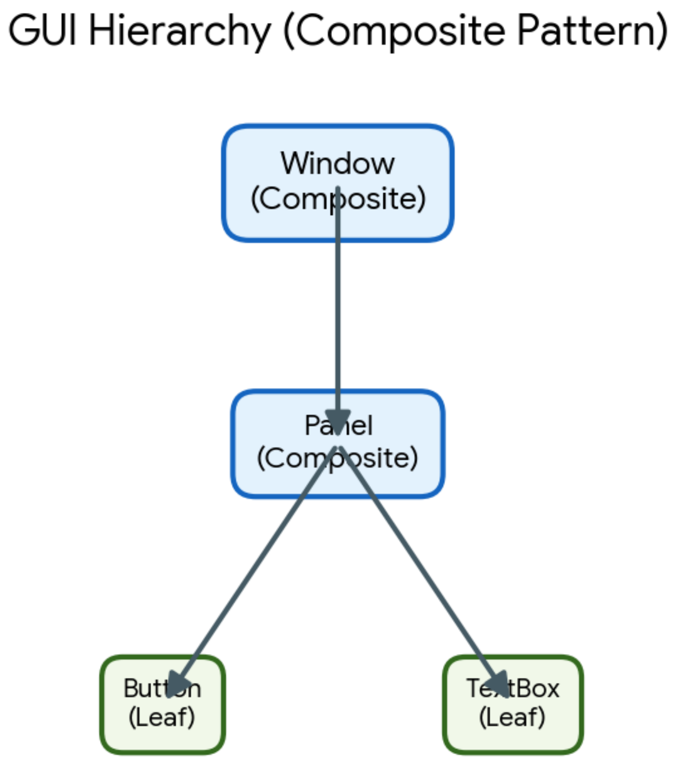
3. GUI System (ប្រព័ន្ធសមាសភាគ UI)

ក្នុងកម្មវិធីកុំព្យូទ័រ សមាសភាគ UI ដូចជាប៊ូតុង (Button) ឬប្រអប់អត្ថបទ (TextBox) ជា Leaf ចំណែកឯ Panel ឬ Window ជា Composite ព្រោះវាអាចផ្ទុកប៊ូតុង និង Panel ផ្សេងទៀតបាន។

Component: VisualElement ឬ Component

Leaf: Button, Label, TextField

Composite: Panel, Frame, Window



Panel អាចផ្ទុក Components ផ្សេងៗបាន។

4. Example Code (Concept)

ឧទាហរណ៍គំនិតក្នុងភាសា C++៖

```
class Component {
public:
    virtual void display( ) = 0;
};
class File : public Component {
public:
    void display( ) {
        cout << "This is File";
```

```
    }
};
class Folder : public Component {
    vector<Component *> items;
public:
    void add( Component * c ) {
        items.push_back( c );
    }
    void display( ) {
        for ( auto item : items ) {
            item->display( );
        }
    }
};
```

ការពន្យល់

+ Component → Base class

+ File → Leaf

+ Folder → Composite

+ Folder អាចមាន File ឬ Folder ផ្សេងទៀត

5. អត្ថប្រយោជន៍ (Advantages)

អត្ថប្រយោជន៍ចម្បងៗរបស់ Composite Pattern រួមមានចំណុចសំខាន់ៗដូចខាងក្រោម៖

១. ភាពងាយស្រួលក្នុងការប្រើប្រាស់ (Uniformity)

Client (អ្នកប្រើប្រាស់) មិនចាំបាច់ខ្វល់ខ្វាយថាវត្ថុដែលខ្លួនកំពុងប្រើជាវត្ថុទោល (Leaf) ឬជា បណ្តុំវត្ថុ (Composite) ឡើយ។ ពួកគេអាចហៅមុខងារដូចគ្នា (ដូចជា draw( ), print( ), getSum( )) ទៅលើគ្រប់វត្ថុទាំងអស់ក្នុងមែកធាង។ វាកាត់បន្ថយការប្រើ if-else ឬ switch-case ដើម្បីឆែកប្រភេទវត្ថុ។

២. ភាពងាយស្រួលក្នុងការបន្ថែមសមាសភាគថ្មី (Extensibility)

អ្នកអាចបន្ថែមប្រភេទ Leaf ថ្មី ឬ Composite ថ្មីៗបានយ៉ាងងាយស្រួល ដោយគ្រាន់តែបង្កើត Class ថ្មីដែលអនុវត្ត (Implement) តាម Component Interface រួចជាស្រេច។ អ្នកមិនចាំបាច់ទៅកែប្រែកូដចាស់ៗ ឬរចនាសម្ព័ន្ធដែលមានស្រាប់នោះទេ។

៣. បង្កើតរចនាសម្ព័ន្ធឋានានុក្រមស្មុគស្មាញ (Hierarchical Structure)

Pattern នេះអនុញ្ញាតឱ្យអ្នកសាងសង់រចនាសម្ព័ន្ធមែកធាងដែលមានភាពស្មុគស្មាញ និងមានច្រើនជាន់ (Recursive) បានយ៉ាងបត់បែន។ ឧទាហរណ៍៖

ក្នុង Folder អាចមាន Folder ហើយក្នុង Folder នោះអាចមាន Folder មួយជាន់ទៀតមិនកំណត់កម្រិត។

៤. កូដមានសណ្តាប់ធ្នាប់ និងងាយស្រួលថែទាំ (Cleaner Code)

ដោយសារវាប្រើគោលការណ៍ Recursion (ការហៅបន្តៗគ្នា) វានាំឱ្យកូដរបស់អ្នកមានភាពសាមញ្ញ។ នៅពេលអ្នកបញ្ជាទៅកាន់វត្ថុមេ (Root) វានឹងបញ្ជាបន្តទៅកាន់កូនចៅទាំងអស់ដោយស្វ័យប្រវត្តិ ដែលធ្វើឱ្យកូដមានលក្ខណៈជា Modular និងងាយស្រួលគ្រប់គ្រង។

6. គុណវិបត្តិ (Disadvantages)

ទោះបីជាវាមានអត្ថប្រយោជន៍ច្រើនក៏ដោយ ក៏ Composite Pattern មានគុណវិបត្តិមួយចំនួនដែលអ្នកគួរដឹង៖

១. ពិបាកក្នុងការកម្រិតប្រភេទសមាសភាគ (Difficulty in Restricting Components)

នេះគឺជាគុណវិបត្តិធំបំផុត។ ដោយសារតែ Composite និង Leaf ប្រើ Interface រួមគ្នា (Component) វាមិនងាយស្រួលទេក្នុងការហាមឃាត់មិនឱ្យ Composite ផ្ទុកប្រភេទ Leaf ណាមួយជាក់លាក់។

ឧទាហរណ៍៖

បើអ្នកចង់ឱ្យ Folder A ផ្ទុកបានតែ Image File មិនឱ្យផ្ទុក Text File នោះអ្នកមិនអាចប្រើ Interface រួមបានដោយងាយស្រួលឡើយ។ អ្នកត្រូវឆែកប្រភេទវត្ថុនៅពេល Runtime (instanceof) ដែលវាធ្វើឱ្យបាត់បង់អត្ថប្រយោជន៍នៃ Polymorphism។

២. ការរចនា Interface មានភាពស្មុំញ៉ាំ (Over-generalized Interface)

ដើម្បីឱ្យ Client ចាត់ទុកគ្រប់យ៉ាងដូចគ្នា Interface រួមត្រូវតែមានមុខងារគ្រប់គ្រាន់សម្រាប់ទាំង Leaf និង Composite។

បញ្ហា៖ មុខងារដូចជា add( ) ឬ remove( ) មិនសមស្របសម្រាប់ Leaf (វត្ថុទោល) ឡើយ។ បើអ្នកដាក់វាចូលក្នុង Interface រួម អ្នកនឹងមានមុខងារដែលមិនបានប្រើ (Useless methods) ក្នុងថ្នាក់ Leaf ឬត្រូវបោះកំហុស (Exception) ចេញមកវិញ។

៣. ភាពស្មុគស្មាញនៃឋានានុក្រម (Complexity of Hierarchy)

នៅពេលដែលរចនាសម្ព័ន្ធមែកធាង (Tree) មានច្រើនជាន់ពេក ឬស្មុគស្មាញខ្លាំង វានឹងធ្វើឱ្យការតាមដានលំហូរកូដ (Debugging) មានការពិបា

ក។ ប្រសិនបើមានបញ្ហានៅចុងមែកធាង (Leaf) ណាមួយ អ្នកត្រូវស្វែងរកកាត់តាម Composite ជាច្រើនជាន់។

៤. បញ្ហាអង្គចងចាំ និងប្រសិទ្ធភាព (Memory and Performance)

ប្រសិនបើរចនាសម្ព័ន្ធមានវត្ថុរាប់ពាន់ ឬរាប់ម៉ឺន វានឹងប្រើប្រាស់អង្គចងចាំ (RAM) ច្រើន។ ការប្រើប្រាស់ Recursion (ការហៅបន្តៗគ្នាពីមេទៅកូន) ដើម្បីគណនាអ្វីមួយ ក៏អាចប៉ះពាល់ដល់ល្បឿនដំណើរការ (Performance) ប្រសិនបើមែកធាងនោះជ្រៅពេក។

7. ពេលណាគួរប្រើ Composite Pattern

អ្នកគួរប្រើ Composite Pattern នៅពេលដែលគម្រោងរបស់អ្នកជួបប្រទះស្ថានភាពដូចខាងក្រោម៖

១. នៅពេលចង់បង្កើតរចនាសម្ព័ន្ធបែបមែកធាង (Tree-like Hierarchies)

ប្រសិនបើទិន្នន័យ ឬវត្ថុ (Objects) ក្នុងកម្មវិធីរបស់អ្នកមានទំនាក់ទំនងបែប "របស់តូចនៅក្នុងរបស់ធំ" ឬ "ផ្នែកនៃ (Part-of)" ដែលអាចរីកសាយបន្តបន្ទាប់គ្នាជាច្រើនជាន់។

ឧទាហរណ៍៖

ប្រព័ន្ធម៉ឺនុយ (Menu -> Sub-menu -> Menu Item) ឬ ប្រព័ន្ធគ្រប់គ្រងឃ្លាំង (Container -> Box -> Product)។

២. នៅពេលចង់ឱ្យ Client ប្រើប្រាស់វត្ថុទោល និងវត្ថុបណ្តុំដោយ "មិនបាច់បែងចែក"

ប្រសិនបើអ្នកចង់ឱ្យអ្នកប្រើប្រាស់កូដ (Client) អាចបញ្ជាការងារទៅលើវត្ថុមួយ (Leaf) ឬក្រុមនៃវត្ថុ (Composite) តាមរយៈ Interface តែមួយ ដោយមិនចាំបាច់សរសេរ if-else ដើម្បីឆែកប្រភេទវត្ថុនោះឡើយ។

គោលបំណង៖ ដើម្បីឱ្យកូដរបស់ Client មានភាពសាមញ្ញ និងស្អាត (Clean Code)។

៣. នៅពេលរចនាសម្ព័ន្ធមានភាពបត់បែនខ្លាំង (Dynamic Structure)

នៅពេលដែលអ្នកមិនដឹងមុនថា មែកធាងរបស់អ្នកនឹងមានប៉ុន្មានជាន់ ឬមានវត្ថុអ្វីខ្លះនៅខាងក្នុង។ Composite Pattern អនុញ្ញាតឱ្យអ្នកបន្ថែម ឬដកសមាសភាគចេញនៅពេលកម្មវិធីកំពុងដំណើរការ (Runtime) បានយ៉ាងងាយស្រួល។

៤. នៅពេលចង់អនុវត្តប្រតិបត្តិការបែប Recursive (Recursive Operations)

នៅពេលអ្នកត្រូវការគណនាអ្វីមួយដែលត្រូវបូកសរុបពីកូនចៅទាំងអស់ឡើងមកលើ។

ឧទាហរណ៍៖

ការគណនា ទំហំសរុប នៃ Folder មួយ (ដែលត្រូវបូកទំហំ Files ទាំងអស់ក្នុងនោះ) ឬ ការគណនា តម្លៃសរុប នៃកន្ត្រកទំនិញដែលមានកញ្ចប់កាដូបញ្ចូលគ្នា។

8. ពេលណាមិនគួរប្រើ Composite Pattern

ទោះបីជា Composite Pattern មានប្រយោជន៍ខ្លាំងសម្រាប់រចនាសម្ព័ន្ធមែកធាង ប៉ុន្តែមានស្ថានភាពខ្លះដែលអ្នក មិនគួរប្រើវា ព្រោះវាអាចធ្វើឱ្យកូដកាន់តែស្មុគស្មាញដោយមិនចាំបាច់៖

១. នៅពេលរចនាសម្ព័ន្ធរបស់អ្នកមានលក្ខណៈសំប៉ែត (Flat Structure)

ប្រសិនបើវត្ថុក្នុងកម្មវិធីរបស់អ្នកមិនមានទំនាក់ទំនងបែប "របស់ក្នុងរបស់" ឬមិនមានឋានានុក្រមច្រើនជាន់ទេ ការប្រើ Pattern នេះនឹងធ្វើឱ្យកូដ

របស់អ្នកសុំញ៉ាំខ្លាំង។ ក្នុងករណីនេះ ការប្រើ List ឬ Array ធម្មតាគឺគ្រប់គ្រាន់ហើយ។

២. នៅពេល Leaf និង Composite មានមុខងារខុសគ្នាដាច់ស្រឡះ

Composite Pattern ដំណើរការទៅបាន លុះត្រាតែ Leaf និង Composite អាចប្រើ Interface រួមគ្នាបាន។ ប្រសិនបើវត្ថុទោលរបស់អ្នកមានមុខងារផ្សេង ហើយវត្ថុបណ្តុំមានមុខងារផ្សេងទៀតដែលមិនអាចប្រៀបបញ្ចូលគ្នាបានទេ ការបង្ខំឱ្យពួកវាប្រើ Interface តែមួយនឹងធ្វើឱ្យកូដមានកំហុសច្រើន។

៣. នៅពេលអ្នកចង់កម្រិតប្រភេទកូនចៅឱ្យតឹងរ៉ឹង (Strict Child Constraints)

ប្រសិនបើអ្នកចង់ឱ្យ Folder A ផ្ទុកបានតែ File ប៉ុណ្ណោះ មិនឱ្យផ្ទុក Folder ផ្សេងទៀតទេ នោះ Composite Pattern មិនមែនជាជម្រើសល្អឡើយ។ ដោយសារគោលការណ៍របស់វាគឺចង់ឱ្យគ្រប់យ៉ាង "ដូចគ្នា" ការព្យាយាមដាក់កម្រិត (Restrict) នឹងធ្វើឱ្យអ្នកត្រូវសរសេរកូដឆែកប្រភេទវត្ថុ (instanceof) ច្រើន ដែលផ្ទុយពីគោលបំណងដើមរបស់ Pattern នេះ។

៤. នៅពេលមែកធាងមានទំហំធំខ្លាំង និងបារម្ភពីល្បឿន (Performance Sensitive)

ការប្រើ Recursion (ការហៅបន្តៗគ្នាពីមេទៅកូន) អាចធ្វើឱ្យកម្មវិធីដើរយឺត ឬអស់អង្គចងចាំ (Stack Overflow) ប្រសិនបើមែកធាងនោះមានរាប់ម៉ឺនជាន់។ ប្រសិនបើកម្មវិធីរបស់អ្នកត្រូវការល្បឿនលឿនបំផុត (High Performance) អ្នកគួរប្រើវិធីសាស្ត្រផ្សេងដូចជា Iterative Approach ជំនួសវិញ។

៥. នៅពេលអ្នកមិនត្រូវការភាពដូចគ្នា (Uniformity Not Required)

ប្រសិនបើ Client តែងតែដឹងមុនថាខ្លួនកំពុងប្រើ File ឬ Folder ហើយមិនដែលមានបំណងចង់បញ្ជាពួកវាដូចគ្នានោះទេ ការប្រើ Composite Pattern គ្រាន់តែជាការបន្ថែមបន្ទុកដល់ការសរសេរកូដប៉ុណ្ណោះ។

Chapter 2: SINGLETON

ក្នុង Object-Oriented Analysis and Design (OOAD) Singleton Pattern គឺជា Creational Design Pattern មួយ ដែលមានគោលបំណងធានាថា Class មួយអាចបង្កើត Object បានតែមួយគត់ (Single Instance) ហើយផ្តល់ global access point សម្រាប់ឲ្យផ្នែកផ្សេងៗក្នុងប្រព័ន្ធអាចប្រើ object នោះបាន។

Singleton Pattern គឺមានសារៈសំខាន់ណាស់ក្នុងការអភិវឌ្ឍន៍ software ព្រោះវាជួយគ្រប់គ្រង resource ដែលមានតម្លៃថ្លៃ ឬត្រូវប្រើរួមគ្នា ដូចជា database connection, logging system និង configuration manager។

និយាយឲ្យសាមញ្ញ៖

Singleton Pattern គឺជា design pattern ដែលធានាថា object មួយបង្កើតបានតែមួយក្នុង system ហើយអាចប្រើពីគ្រប់ទីកន្លែងបាន។

1. និយមន័យ Singleton

Singleton Pattern គឺជា design pattern ដែលគ្រប់គ្រង object creation mechanism ដើម្បីធានាថា class មួយមាន instance តែមួយគត់ ហើយអនុញ្ញាតឲ្យ objects ផ្សេងៗក្នុងកម្មវិធីអាច access instance នោះបានដោយងាយស្រួល។

លក្ខណៈសំខាន់ៗរបស់ Singleton Pattern មានដូចជា៖

Class មាន instance តែមួយ

Instance នោះអាច access បានពីគ្រប់ module ក្នុង system

Class គ្រប់គ្រងការបង្កើត object ដោយខ្លួនឯង

2. ប្រវត្តិ និងសារៈសំខាន់

Singleton Pattern ជាមួយនឹង design patterns ផ្សេងៗ ត្រូវបានពិពណ៌នាដំបូងក្នុងសៀវភៅ Design Patterns: Elements of Reusable Object-Oriented Software ដែលសរសេរដោយ Erich Gamma, Richard Helm, Ralph Johnson និង John Vlissides (ដែលគេហៅថា Gang of Four)។

Singleton Pattern ត្រូវបានប្រើយ៉ាងទូលំទូលាយក្នុង software architecture ដើម្បីគ្រប់គ្រង shared resources និងបង្កើន system consistency។

3. គោលបំណង (Purpose)

គោលបំណងសំខាន់របស់ Singleton Pattern គឺ៖

- ការពារមិនឲ្យបង្កើត object ច្រើន

- គ្រប់គ្រង resource សំខាន់
- ផ្តល់ global access
- កាត់បន្ថយ memory usage

1. **ការពារមិនឲ្យបង្កើត object ច្រើន**

    ក្នុង system មួយ ប្រសិនបើមាន object ច្រើនអាចបង្កើត conflict ឬ resource waste។

2. **គ្រប់គ្រង resource សំខាន់**

    Resource ដូចជា database connection ឬ configuration គួរតែមានតែមួយ។

3. **ផ្តល់ global access**

    Modules ទាំងអស់ក្នុង system អាចប្រើ instance ដូចគ្នា។

4. **កាត់បន្ថយ memory usage**

    ការប្រើ instance តែមួយធ្វើឲ្យ system ប្រើ memory តិច។

4. រចនាសម្ព័ន្ធ (Structure)

Singleton Pattern មាន Class តែមួយ ប៉ុន្តែមានលក្ខណៈពិសេស ៣ ដែលធ្វើឲ្យវាដំណើរការបាន។

1. **Private Constructor**

    Constructor ត្រូវបានកំណត់ជា private ដើម្បីបញ្ឈប់ការបង្កើត object ពី class ផ្សេង។

    នេះមានន័យថា object អាចបង្កើតបានតែ ខាងក្នុង class Singleton ប៉ុណ្ណោះ។

2. **Static Instance**

    Singleton class មាន static variableមួយសម្រាប់រក្សាទុកinstance។

Static variable មានសារៈសំខាន់ព្រោះវាជា variable ដែលមានតម្លៃ shared សម្រាប់ class ទាំងមូល។

3. Public Static Method

Method ដែលគេហៅថា getInstance( ) ត្រូវបានប្រើសម្រាប់ return instance តែមួយ។

Method នេះពិនិត្យថា instance មានឬអត់ មុនពេលបង្កើត object។

5. UML Class Diagram របស់ Singleton

- ការពន្យល់ Diagram

- Singleton Class
- ជាថ្នាក់ដែលគ្រប់គ្រង instance។
- instance
- ជាវ៉ារីយ៉ាប៊ែល static ដែលរក្សាទុក object តែមួយ។
- getInstance( )
- method សម្រាប់ return instance។

- operation( )
- function ផ្សេងៗដែល Singleton អនុវត្ត។

6. ដំណើរការរបស់ Singleton Pattern

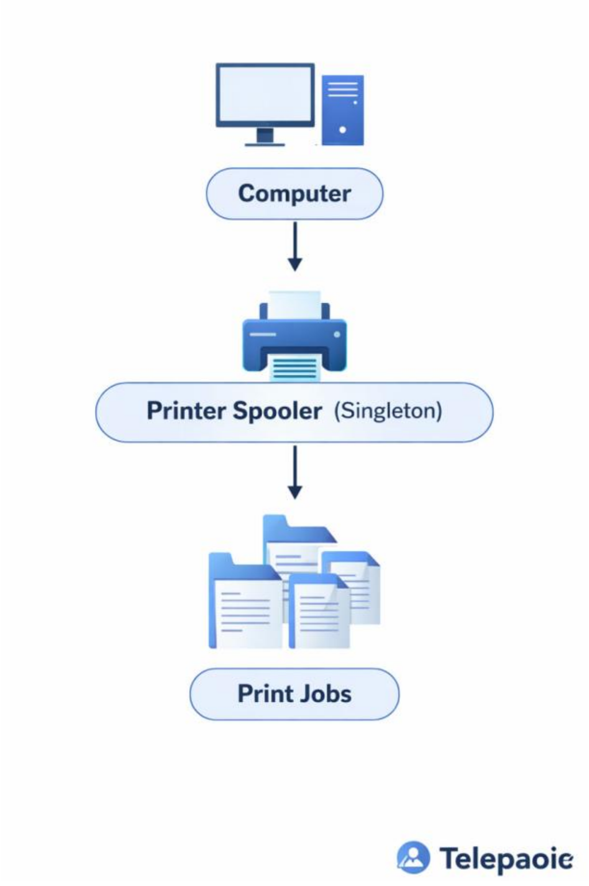
▪ ដំណើរការរបស់ Singleton Pattern មានដូចខាងក្រោម៖

- System ហៅ method getInstance( )
- Method ពិនិត្យថា instance មានឬអត់
- បើ instance មិនទាន់មាន → បង្កើត object ថ្មី
- បើ instance មានហើយ → return object ដដែល
- Modules ផ្សេងៗប្រើ instance នោះ

7. ឧទាហរណ៍ជាក់ស្តែង (Real-world Examples)

1. Printer Spooler

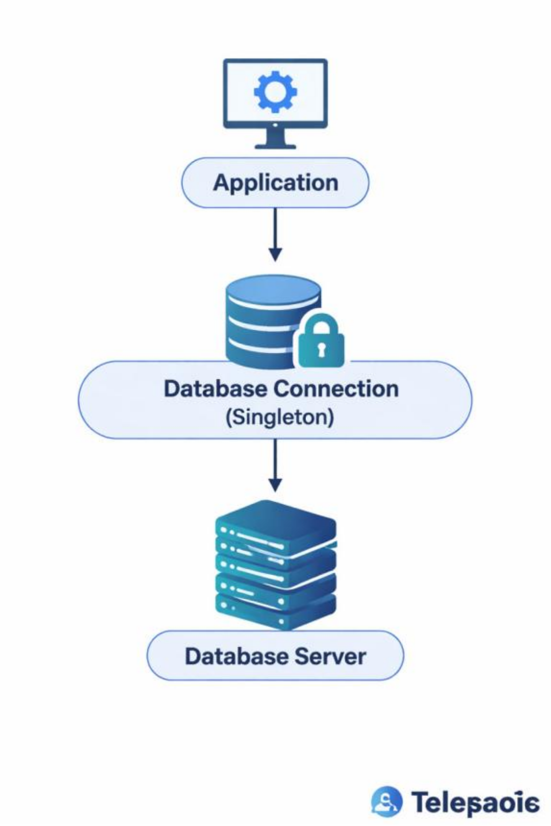
ក្នុង computer មួយ មាន printer spooler តែមួយ ដែលគ្រប់គ្រង print jobs ទាំងអស់។



ប្រសិនបើមាន spooler ច្រើន អាចបង្កើត conflict ក្នុង printing process។

2. Database Connection

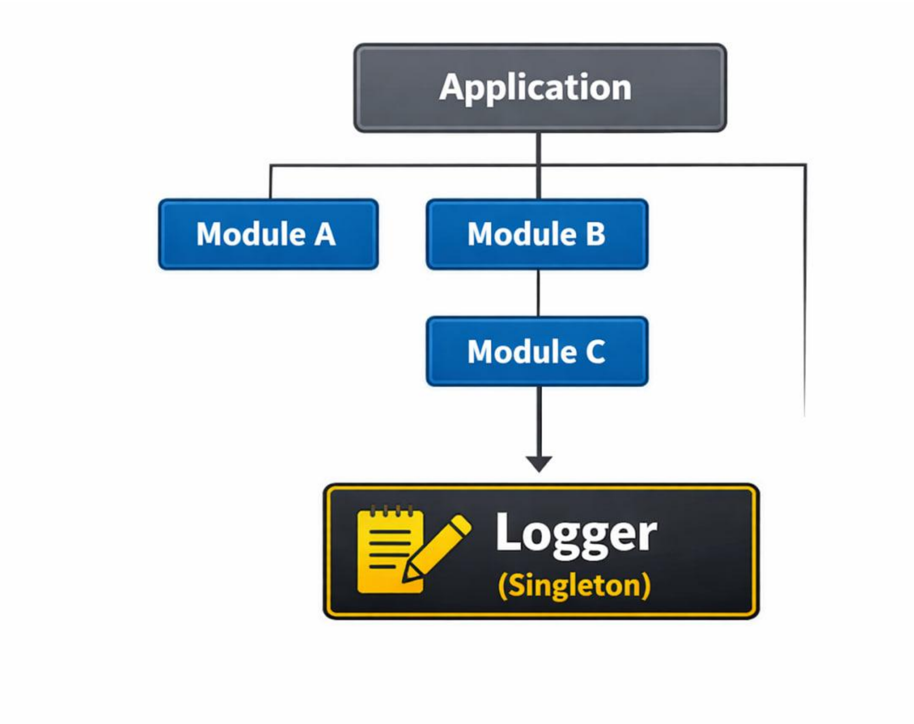
Application មួយភាគច្រើនប្រើ database connection តែមួយ។



បើបង្កើត connection ច្រើន អាចបង្កើត performance problem។

3. Logging System

កម្មវិធីធំៗប្រើ logger តែមួយ សម្រាប់កត់ត្រា system activities។



Logger តែមួយធ្វើឲ្យ logs មាន consistency។

8. Example Code (C++)

```
class Singleton {
private:
    static Singleton * instance;
    Singleton( ) {}
public:
    static Singleton * getInstance( ) {
        if ( instance == NULL ) {
            instance = new Singleton( );
        }
        return instance;
    }
    void showMessage( ) {
        cout << "Singleton Instance";
```

```
    }
};
```

- Singleton * Singleton::instance = NULL;

- ការពន្យល់ Code

- instance → static variable សម្រាប់រក្សាទុក object
- private constructor → បញ្ឈប់ការបង្កើត object ពីក្រៅ
- getInstance( ) → method សម្រាប់ return instance

9. អត្ថប្រយោជន៍ (Advantages)

Singleton Pattern មានអត្ថប្រយោជន៍សំខាន់ៗជាច្រើន ដែលធ្វើឲ្យវាត្រូវបានប្រើប្រាស់យ៉ាងទូលំទូលាយក្នុង software development។

1. **គ្រប់គ្រង instance បានល្អ (Controlled Instance Creation)**
   - Singleton ធានាថា Class មួយមាន object តែមួយគត់
   - មិនអាចបង្កើត object ច្រើនដោយចៃដន្យបានទេ
   - ជួយបញ្ឈប់បញ្ហា duplicate objects

   ឧទាហរណ៍៖

   Database connection បើមានច្រើន អាចបង្កើត conflict ឬ error

2. **កាត់បន្ថយ resource usage (Memory & Resource Efficiency)**
   - ព្រោះមាន instance តែមួយ → ប្រើ memory តិច
   - មិនចាំបាច់បង្កើត object ថ្មីរាល់ពេល
   - សមស្របសម្រាប់ resource ថ្លៃៗ (expensive resources)

   ឧទាហរណ៍៖

   - Database connection
   - File handler
   - Network socket

3. **Global access**

- Singleton ផ្តល់ global access point
- Module ណាក៏អាចហៅ getInstance( ) ដើម្បីប្រើ object ដូចគ្នា

▪ អត្ថប្រយោជន៍៖

- កូដងាយស្រួលប្រើ
- មិនចាំបាច់ pass object ជា parameter ទៅគ្រប់ function

4. Consistency
    - ព្រោះមាន instance តែមួយ → data មានភាពដូចគ្នាទូទាំង system
    - មិនមាន data mismatch រវាង modules

▪ ឧទាហរណ៍៖

- Logger តែមួយ → logs មាន format និង source ដូចគ្នា

5. Centralized Control (គ្រប់គ្រងកណ្តាល)
    - Resource ទាំងអស់ត្រូវបានគ្រប់គ្រងតែមួយកន្លែង
    - ងាយស្រួល maintain និង update

▪ ឧទាហរណ៍៖

- Configuration Manager
- → ប្តូរតែម្តង មានប្រសិទ្ធិភាពលើ system ទាំងមូល

6. Lazy Initialization (Optional)
    - Object ត្រូវបានបង្កើត នៅពេលត្រូវការ
    - មិនបង្កើតតាំងពីដំបូង → save performance

10. គុណវិបត្តិ (Disadvantages)

បើទោះបីមានអត្ថប្រយោជន៍ច្រើន ក៏ Singleton មានគុណវិបត្តិដែលសំខាន់:

1. Global state problem

Singleton ជា global object → អាចបង្កើត dependency ខ្លាំង

- Modules ច្រើនពឹងផ្អែកលើ Singleton មួយ

▪ បញ្ហា៖

- កែប្រែ Singleton → អាចប៉ះពាល់ system ទាំងមូល
- Hard to track bugs

2. Tight Coupling (ភាពភ្ជាប់ខ្លាំង)

- Classes ផ្សេងៗពឹងផ្អែកលើ Singleton
- បង្កើត coupling ខ្លាំង → design មិន flexible

▪ ផលវិបាក៖

- ពិបាក refactor
- ពិបាក reuse code

3. Hard to Unit Testing

- Singleton មាន state shared → test មួយអាចប៉ះពាល់ test ផ្សេង
- មិនងាយ mock ឬ replace

▪ ឧទាហរណ៍៖

- Test A ប្តូរ data → Test B ទទួលបាន data ដដែល → error

4. Thread Safety Issues

- ក្នុង multi-thread environment អាចមាន race condition

▪ បញ្ហា៖

- Threads 2 អាចបង្កើត instance 2 ដោយចៃដន្យ

▪ ដំណោះស្រាយ៖

- Use mutex / lock
- Use thread-safe implementation

5. Hidden Dependencies

- Singleton ត្រូវបាន access global → dependency មិនច្បាស់

- កូដមើលទៅមិនសូវច្បាស់ថា class ត្រូវការ resource អ្វី

6. Scalability Issues

- Singleton អាចក្លាយជា bottleneck
- Threads ច្រើនប្រើ instance តែមួយ → performance អាចធ្លាក់

7. Violates Single Responsibility Principle (SRP)

- Class មួយមានភារកិច្ច ២៖
  - Business logic
  - Control instance

នេះធ្វើឲ្យ design មិនស្អាត

11. ពេលណាគួរប្រើ Singleton Pattern

Singleton មិនគួរប្រើគ្រប់ករណីទេ ❗ ត្រូវប្រើនៅពេលសមស្រប

1. ត្រូវការតែ instance មួយ

System ត្រូវការតែ object មួយគត់

▪ ឧទាហរណ៍៖

- Logger
- Config Manager
- Cache Manager

2. Shared Resource

- Resource ត្រូវប្រើរួមគ្នា

▪ ឧទាហរណ៍៖

- Database connection
- Thread pool
- File system manager

3. Global Access Required

- Modules ច្រើនត្រូវការប្រើ object ដូចគ្នា

- ប្រើ Singleton → ងាយ access

4. **Expensive Object Creation**
   - Object បង្កើតយឺត ឬចំណាយ resource ច្រើន
   - Singleton ជួយ reuse object

5. **Need Consistent State**
   - Data ត្រូវតែដូចគ្នាទូទាំង system
   - ឧទាហរណ៍៖
   - Application settings
   - Logging system

12. ពេលណាមិនគួរប្រើ Singleton Pattern

1. **ត្រូវការបង្កើត object ច្រើន**

   System ត្រូវការច្រើន instances → Singleton មិនសម

2. **ត្រូវការ flexibility ខ្ពស់**

   System ត្រូវការប្តូរ implementation ញឹកញាប់

3. **Testing មានសារៈសំខាន់**

   Project ត្រូវការធ្វើ unit test ច្រើន

4. **Multi-thread heavy system**

   ប្រសិនបើមិនគ្រប់គ្រង thread safety → អាចមាន bug

   - Singleton Pattern គួរប្រើនៅពេល៖

- System ត្រូវការតែ object មួយ
- Resource ត្រូវប្រើរួមគ្នា
- Modules ត្រូវការប្រើ configuration ដូចគ្នា
- System ត្រូវការគ្រប់គ្រង resource សំខាន់

Chapter 3: Observer Pattern

ក្នុង Object-Oriented Analysis and Design (OOAD) Observer Pattern គឺជា Behavioral Design Pattern មួយ ដែលកំណត់ ទំនាក់ទំនងមួយទៅច្រើន (One-to-Many Relationship) រវាង objects។

មាន object មួយហៅថា Subject និង objects ជាច្រើនហៅថា Observers។

នៅពេល Subject មានការផ្លាស់ប្តូរ វានឹង ជូនដំណឹង (notify) ទៅ observers ទាំងអស់ដោយស្វ័យប្រវត្តិ។

និយាយឲ្យសាមញ្ញ៖

Observer Pattern អនុញ្ញាតឲ្យ objects ជាច្រើន ទទួលព័ត៌មាន update ពី object មួយ នៅពេលមានការផ្លាស់ប្តូរ។

1. **និយមន័យ** (Definition)

   Observer Pattern គឺជា design pattern ដែលកំណត់ទំនាក់ទំនង one-to-many រវាង objects ដូច្នេះនៅពេល object មួយផ្លាស់ប្តូរ state នោះ objects ផ្សេងទៀតដែលពាក់ព័ន្ធនឹងត្រូវបាន notify និង update ដោយស្វ័យប្រវត្តិ។ ក្នុងភាសា C#, pattern នេះជួយកាត់បន្ថយការភ្ជាប់ជិតស្និទ្ធ (loose coupling) ដោយអនុញ្ញាតឱ្យ observers អាច subscribe ទៅ subject ដោយមិនចាំបាច់ឲ្យ subject ដឹងពីប្រភេទពិតរបស់ observers ទាំងនោះ។

2. **គោលបំណង (Purpose)**

   ➢ គោលបំណងសំខាន់របស់ Observer Pattern គឺ៖

   - បង្កើត communication រវាង objects
   - កាត់បន្ថយ tight coupling
   - ឲ្យ objects អាច update ព័ត៌មានដោយស្វ័យប្រវត្តិ

វាត្រូវបានប្រើនៅពេលដែល system ត្រូវការឲ្យ objects ជាច្រើន react ទៅនឹងការផ្លាស់ប្តូរ។

3. រចនាសម្ព័ន្ធ (Structure)

➢ Observer Pattern មាន ៤ Components សំខាន់៖

- Subject
- Observer
- ConcreteSubject
- ConcreteObserver

1. Subject

Subject គឺជា object ដែល រក្សាទុក state និង ជូនដំណឹងទៅ observers នៅពេល state ផ្លាស់ប្តូរ។

- Subject មាន method ដូចជា៖
- attach( )
- detach( )
- notify( )

2. Observer

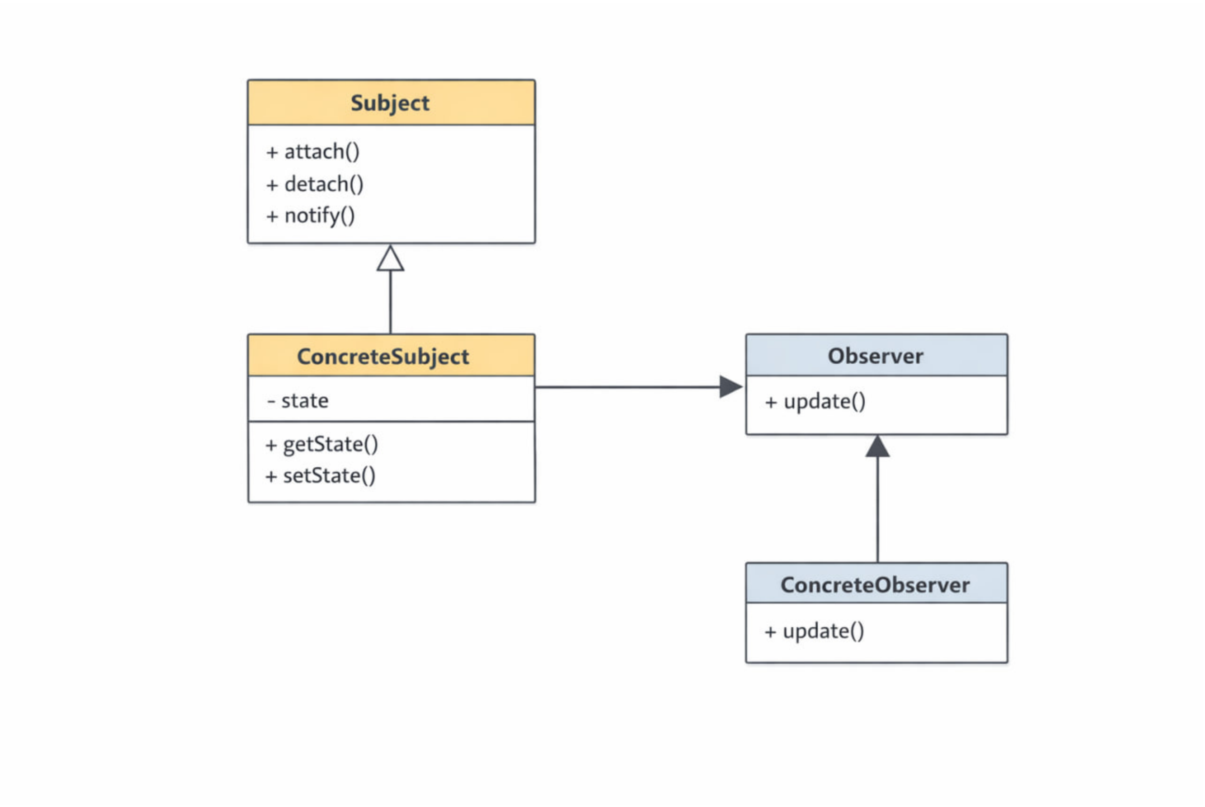
Observer គឺជា interface ឬ abstract class ដែលកំណត់ method update( ) សម្រាប់ observers ទទួល notification។

3. ConcreteSubject

4. ConcreteObserver

ConcreteSubject គឺជា class ដែលអនុវត្ត Subject និងរក្សាទុក state ពិតប្រាកដ។
នៅពេល state ផ្លាស់ប្តូរ វានឹង notify observers។

4.UML Class Diagram របស់ Observer Pattern



5. ដំណើរការរបស់ Observer Pattern

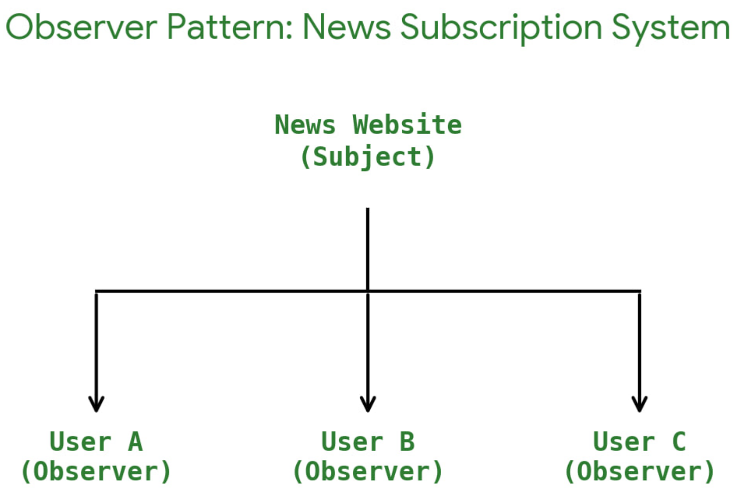
➢ ដំណើរការរបស់ Observer Pattern មានដំណាក់កាលដូចខាងក្រោម៖

- Observer ចុះឈ្មោះ (subscribe) ជាមួយ Subject
- Subject រក្សាទុក observers ក្នុង list
- នៅពេល state ផ្លាស់ប្តូរ
- Subject ហៅ method notify( )
- Observers ទាំងអស់ទទួល update( )

6. ឧទាហរណ៍ជាក់ស្តែង (Real-world Examples)

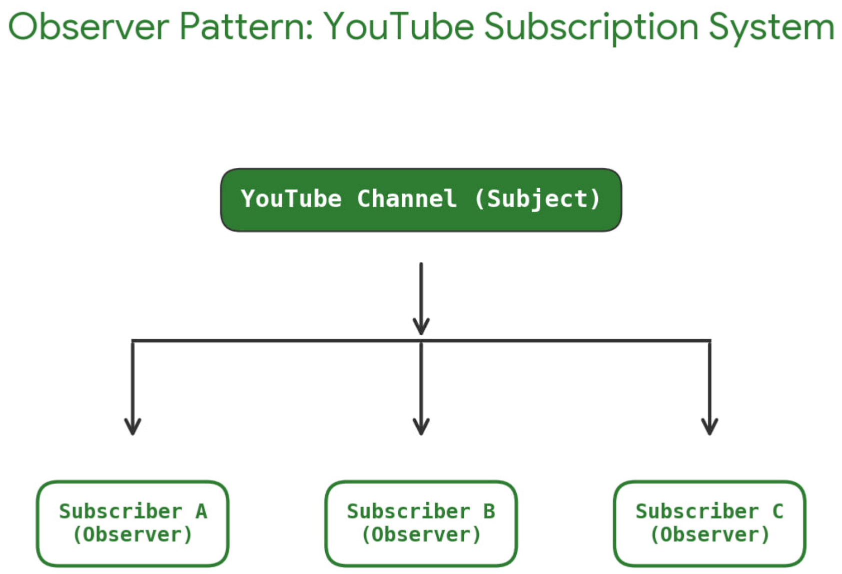
1. **News Subscription**

អ្នកប្រើ subscribe news website។



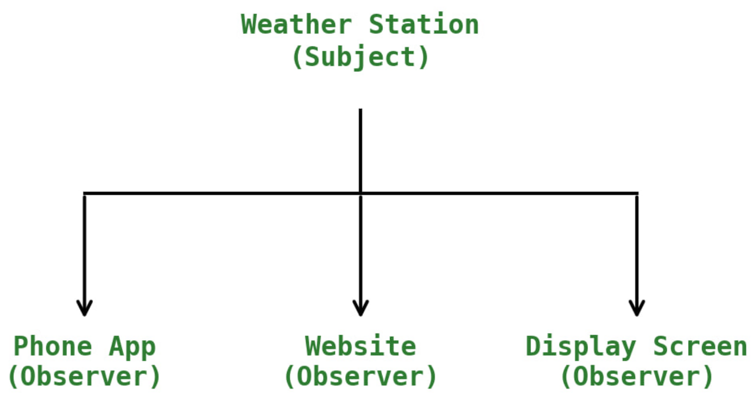
ពេលមាន news ថ្មី → users ទាំងអស់ទទួល notification។

2. YouTube Subscription



3. Weather Monitoring System

Observer Pattern: Weather Monitoring System



When weather changes, all devices update automatically.

ពេល temperature ផ្លាស់ប្តូរ → displays update។

7. Example Code (C++)

```
class Observer {
public:
    virtual void update( ) = 0;
};
class Subject {
public:
    virtual void attach( Observer * o ) = 0;
    virtual void detach( Observer * o ) = 0;
    virtual void notify( ) = 0;
};
```

8. អត្ថប្រយោជន៍ (Advantages)

Observer Pattern មានអត្ថប្រយោជន៍សំខាន់ៗដែលធ្វើឲ្យវាពេញនិយមក្នុង systems ដែលមាន event-driven behavior។

- Loose Coupling (បន្ថយការភ្ជាប់គ្នាខ្លាំង)
- Automatic Update (Update ដោយស្វ័យប្រវត្តិ)
- Scalability (អាចពង្រីកបានងាយ)
- Reusability (ប្រើឡើងវិញបាន)
- Support Event-Driven Programming
- Dynamic Relationship
- Separation of Concerns

1. **Loose Coupling(បន្ថយការភ្ជាប់គ្នាខ្លាំង)**
   - Subject និង Observer មិនចាំបាច់ដឹងលម្អិតគ្នា
   - Subject គ្រាន់តែដឹងថាមាន observers មួយចំនួន (interface) ប៉ុណ្ណោះ

   ▪ អត្ថប្រយោជន៍៖
   - ងាយស្រួលកែប្រែ system
   - មិនប៉ះពាល់ code ផ្នែកផ្សេង

   ▪ ឧទាហរណ៍៖

   YouTube channel មិនចាំបាច់ដឹងថា subscriber ជានរណា។

2. **Automatic Update (Update ដោយស្វ័យប្រវត្តិ)**
   - នៅពេល Subject ផ្លាស់ប្តូរ → observers ទាំងអស់ update ដោយ auto
   - មិនចាំបាច់សរសេរ code ដោយដៃសម្រាប់ notify

   ▪ អត្ថប្រយោជន៍៖
   - កាត់បន្ថយ human error
   - System មានភាព reactive (real-time)

3. Scalability (អាចពង្រីកបានងាយ)
    - អាចបន្ថែម observers ថ្មីបាន ដោយមិនប៉ះពាល់ Subject
  - ឧទាហរណ៍៖
    - បន្ថែម Mobile App observer ថ្មី
    - → មិនបាច់កែ Weather Station
4. Reusability (ប្រើឡើងវិញបាន)
    - Observer interface អាច reuse ក្នុង project ផ្សេងៗ
    - ConcreteObserver អាចប្រើជាមួយ Subject ផ្សេង
5. Support Event-Driven Programming
    - Observer Pattern គឺជា base សម្រាប់ event system
    - ប្រើក្នុង GUI, notification system, real-time apps
  - ឧទាហរណ៍៖
    - Button click event
    - Message notification
6. Dynamic Relationship

   Observers អាច:
    - subscribe (attach) ពេលណាក៏បាន
    - unsubscribe (detach) ពេលណាក៏បាន
    - មាន flexibility ខ្ពស់
7. Separation of Concerns
    - Subject → គ្រប់គ្រង state
    - Observer → គ្រប់គ្រង reaction

   ធ្វើឲ្យ design ស្អាត និងងាយ maintain

9. គុណវិបត្តិ (Disadvantages)

ទោះបីមានអត្ថប្រយោជន៍ច្រើន ក៏ Observer Pattern មានបញ្ហាដែលត្រូវប្រុងប្រយ័ត្ន

1. **Performance Problem (បញ្ហាល្បឿន)**
   - បើមាន observers ច្រើន → notify ទាំងអស់អាចយឺត
   - បញ្ហា៖
   - CPU usage កើន
   - Delay ក្នុង system
   - ឧទាហរណ៍៖
   - 1000 observers → notify 1000 ដង
2. **Unnecessary Updates (Update មិនចាំបាច់)**
   - Observers ទទួល update ទោះបីមិនចាំបាច់
   - បញ្ហា៖
   - Waste resource
   - Performance ធ្លាក់
   - ឧទាហរណ៍៖
   - UI update ទោះបី data មិនប្រើ
3. **Debugging Difficult (ពិបាក debug)**
   - Flow មិនត្រង់ (indirect communication)
   - មិនដឹងថា observer ណាធ្វើឲ្យ error
   - បញ្ហា៖
   - Stack trace ពិបាកអាន
   - Bug tracking ពិបាក
4. **Memory Leak Risk**

- **បើ observer មិន detach → Subject នៅតែរក្សាទុក reference**
  - **បញ្ហា៖**

- **Memory មិនត្រូវបាន free**
- **App អាច crash**

5. Order of Notification (លំដាប់មិនច្បាស់)
   - មិនធានាលំដាប់ notify observers
   - បញ្ហា៖
   - Observer A ត្រូវ run មុន B ប៉ុន្តែមិនប្រាកដ
   - អាចបង្កើត inconsistent result
6. Cascade Updates (Chain Reaction)
   - Observer មួយ update → បង្កើត update ថ្មី → loop
   - បញ្ហា៖
   - Infinite loop
   - System crash
7. Tight Coupling (Indirect)
   - ទោះបី loose coupling ក៏ដោយ
   - → នៅតែមាន dependency តាម interface

10. ពេលណាគួរប្រើ Observer Pattern

Observer Pattern គួរប្រើនៅពេលដែល system មាន event-driven behavior ឬ data change propagation

1. One-to-Many Dependency
   - Object មួយ (Subject) មាន objects ច្រើនពឹងផ្អែក
   - ឧទាហរណ៍៖
   - Weather system → apps ច្រើន
   - News system → users ច្រើន
2. Need Automatic Notification

- ត្រូវការឲ្យ observers update ដោយ auto
- ឧទាហរណ៍៖
- Stock price change
- Chat app notification

3. Event-Driven System

- System ប្រើ events ជាចម្បង
- ឧទាហរណ៍៖
- GUI (button click)
- Game events
- Real-time dashboard

4. Dynamic Subscription

- Objects អាច join/leave system
- ឧទាហរណ៍៖
- User subscribe/unsubscribe
- Plugin system

5. Decoupled Architecture

- ចង់បំបែក logic រវាង components
- ឧទាហរណ៍៖
- Microservices
- Modular system

11. ពេលណាមិនគួរប្រើ Observer Pattern

1. System តូច និងសាមញ្ញ

មិនចាំបាច់ប្រើ pattern → complexity កើន

2. Performance Sensitive System

Observers ច្រើន → system យឺត

3. **Need Strict Control Flow**

    ប្រសិនបើត្រូវការលំដាប់ច្បាស់

→ Observer មិនសម

4. **Memory Management សំខាន់**

    បើមិនគ្រប់គ្រង detach → memory leak